RiscBasic

Reference Manual

A BASIC for RISC-V microcontrollers and FPGA soft cores, compiled directly to machine code

Compiler version 0.36 · Language specification Draft 0.29

13 November 2026 (7 AG)

Language design: Antti Lukats (Identor #1)

Compiler: Claude (Identor #9)

License: MIT
Copyright 7AG (2026) MicroFPGA UG(h)
https://github.com/micro-FPGA/RiscBasic

Contents

1 Introduction 4
1.1 Design goals 4
1.2 The toolchain 4
1.3 Conventions in this manual 4
2 Getting started 5
2.1 Requirements 5
2.2 A first program 5
2.3 Command line 5
2.4 Running a program 6
3 Program structure and syntax 7
3.1 Layout of a program 7
3.2 Lines, statements and comments 7
3.3 Names 7
3.4 Numbers and characters 7
4 Target and memory 8
4.1 TARGET 8
4.2 CLOCK and CPI 8
4.3 CODE, RAM and STACK 8
4.4 Startup 8
5 Include files and conditional compilation 9
5.1 .include 9
5.2 #IF, #ELSEIF, #ELSE, #ENDIF 9
5.3 Defining constants on the command line 9
6 Objects: registers, variables and constants 10
6.1 CPU registers 10
6.2 Variables: DIM 10
6.3 Constants: CONST 11
6.4 Aliases: ALIAS 11
7 Devices 12
7.1 Registers, bits and fields 12
7.2 Set and clear registers 12
7.3 Active-low bits and fields (NOT) 12
7.4 Write-only registers (WRITEONLY) 13
8 Memory access: MEM 14
8.1 Cost of an access 14
9 Assignment and expressions 15
9.1 The assignment rule 15
9.2 ATOMIC 15
9.3 Operators 15
10 Shift and rotate commands 16
11 Strings 17
11.1 Buffers and string expressions 17
11.2 String functions 17
11.3 Comparing strings 18
11.4 String functions and parameters 18
12 Control flow 19
12.1 IF 19
12.2 Loops 19

12.3 SELECT CASE 19
12.4 GOTO, labels and END 20
13 Subroutines, functions and interrupts 21
13.1 SUB and FUNCTION 21
13.2 Calling convention 21
13.3 Interrupts: ON IRQ 21
13.4 Board initialisation: SUB BoardInit 22
14 Timing and waiting 23
15 Console input and output 24
15.1 CONSOLE 24
15.2 PRINT 24
15.3 INPUT 24
15.4 LINE INPUT 24
15.5 READ and WRITE 25
16 Inline assembly 26
17 Compiler output and debugging 27
17.1 Output files 27
17.2 The listing file 27
17.3 ELF and DWARF 27
17.4 Source-level debugging 27
17.5 Compressed instructions and code size 28
18 Complete example 29
Appendix A Keyword reference 30
Appendix B Reserved words 32
Appendix C Implementation notes and limits 33
C.1 Runtime routines 33

1 Introduction

RiscBasic is a BASIC-like language for RISC-V processors with small code memories: soft cores in FPGAs and small hard cores, typically with **8 KB of code memory or less**. The compiler translates each BASIC line directly into RISC-V machine code in a single pass. There is no assembler step, no object file and no linker.

The language follows one principle: **as easy as BASIC, as efficient as assembler**. Hardware is accessed through *objects*: CPU registers, bits and fields of registers, variables, memory and memory-mapped device registers. Every hardware action can be written as one assignment, `DEST = SOURCE`, and the compiler picks the shortest correct instruction sequence for the pair of object kinds.

1.1 Design goals

- Single-pass, direct translation from BASIC to machine code.
- Predictable code: every compile writes a listing that shows each source line with the addresses, machine code and disassembly generated for it.
- Small runtime: helper routines (multiply, divide, print, string handling) are included only when a program uses them, each at most once. This *smart inclusion* replaces the linker.
- Standard output files: raw binary, Intel HEX, `$readmemh` memory files for FPGA block RAM, and ELF with symbols and DWARF line information for source-level debugging.
- The compiler runs on a PC and is written in Python 3, with no dependencies.

Not part of the language: floating point, dynamic memory, heap-allocated strings and an operating system. Text is handled in fixed-size buffers (chapter 11).

1.2 The toolchain

File	Purpose
`riscbasic.py`	The compiler. Reads a `.bas` file and writes `.bin`, `.hex`, `.mem`, `.lst` and `.elf`.
`rbsim.py`	A small RV32I/M/C/Zbb instruction-set simulator, used for testing compiled programs.
`mem2elf.py`	Converts a `.mem` file into an ELF executable when only the memory image is available.
`tests/run_tests.py`	The test suite: compiles test programs for several targets and runs them in the simulator.
`tests/check_gnu.py`	Cross-checks the compiler output against GNU binutils (objdump, objcopy, readelf).
`RiscBasic-Spec.md`	The language specification. This manual describes the same language for programmers.

1.3 Conventions in this manual

- Keywords are written in upper case (`PRINT`, `DEVICE`), but RiscBasic is **not case sensitive**: `print`, `Print` and `PRINT` are the same. Names are case-insensitive too.
- Square brackets in syntax lines mark optional parts: `FOR var = a TO b [STEP s]`. In program text, `[ ]` is used for bit fields and memory addresses.
- Generated code is shown with RISC-V ABI register names, as the listing shows it.

2 Getting started

2.1 Requirements

The compiler needs Python 3 and nothing else. Copy `riscbasic.py` (and optionally `rbsim.py` and `mem2elf.py`) to any directory. A GNU RISC-V toolchain is not needed to compile; it is useful for debugging with GDB.

2.2 A first program

```
TARGET RV32IC
CLOCK  25_000_000
CODE   AT &H0000 SIZE 8K
RAM    AT &H8000 SIZE 2K

DEVICE UART0 AT &H4000_1000
  REG DATA   AT +0 : 8
  REG STATUS AT +4 READONLY
  BIT TXREADY = STATUS.0
END DEVICE

CONSOLE UART0
PRINT "Hello, RISC-V!", 13, 10
```

The first four lines describe the target: the instruction set, the clock frequency, and where code and RAM are. The `DEVICE` block describes a UART: its base address, its registers and the bit that says the transmitter is ready. `CONSOLE` makes it the default device for `PRINT`.

Compile it:

```
python riscbasic.py hello.bas
```

The compiler reports the code and RAM used:

```
hello.bas: 124 bytes code (of 8192), 0 bytes RAM variables
```

and writes these files next to the source:

File	Contents
`hello.bin`	Raw binary image, starting at the `CODE` address.
`hello.hex`	Intel HEX.
`hello.mem`	One 32-bit word per line in hex, for Verilog/VHDL `$readmemh` block-RAM initialisation.
`hello.lst`	Listing: each source line, followed by the address, machine code and disassembly generated for it, and a size summary.
`hello.elf`	ELF32 RISC-V executable with symbols and DWARF line information, for GDB, OpenOCD, XSDB and simulators.

2.3 Command line

```
python riscbasic.py [options] source.bas
```

Option	Effect
`-o NAME`, `--output NAME`	Base name of the output files (default: the source name). `-o build/x` writes `build/x.bin` and so on.
`--no-lst`	Do not write the `.lst` listing.
`--no-elf`	Do not write the `.elf` file.
`--no-debug`	Write the `.elf` without DWARF line information.
`-I DIR`	Also search `DIR` for `.include` files (after the directory of the including file). May be given more than once.

Option	Effect
`-D NAME[=VALUE]`	Define `CONST NAME` before the first line, for conditional compilation (default value 1). Example: `-D DEBUG -D BOARD=2`.
`-h`, `--help`	Show the options.

Errors and warnings name the file and the line: `lib/uart0.bas:18: error: ...`. An error stops the compile; no output files are written.

2.4 Running a program

- **In an FPGA:** use the `.mem` file to initialise the code memory (for example with `$readmemh`), or load the `.elf` or `.bin` with your debugger or boot loader.
- **In the simulator:** `python rbsim.py hello.bin` runs the image and shows the UART output.
- **In a debugger:** load `hello.elf` into GDB or XSDB; see section 17.4.

3 Program structure and syntax

3.1 Layout of a program

A RiscBasic program is one source file, plus any files it includes. It usually has this order:

1. Target and memory declarations: `TARGET`, `CLOCK`, `CODE`, `RAM`, optionally `STACK` and `CPI` (chapter 4).
2. Includes and conditional compilation where needed (chapter 5).
3. Device declarations, aliases, constants and variables (chapters 6 and 7).
4. The program statements. `SUB`, `FUNCTION` and `ON IRQ` blocks may be placed anywhere between statements; the compiler jumps around them.

`RAM` must be declared before the first statement. Declarations (`CONST`, `DIM`, `DEVICE`, `ALIAS`, `CONSOLE`, ...) can also appear between statements. The program starts at the first statement; when it reaches its last line or an `END` statement, it stops in an endless loop.

3.2 Lines, statements and comments

- One statement per line, or several separated by `:`, as in `s = 0 : FOR i = 1 TO 4 : s = s + i : NEXT`. Blocks may open and close on the same line.
- A `:` inside `( )` or `[ ]`, as in `R10[7:4]`, and the width in `REG DATA AT +0 : 8` are not separators.
- Comments start with `'` or `REM` and run to the end of the line. `REM` after a `:` comments out the rest of the line.
- `name:` at the start of a line defines a label for `GOTO`, unless `name` is a `SUB` or `FUNCTION`.
- `LET` is optional: `LET x = 1` is the same as `x = 1`.

3.3 Names

Names start with a letter or _ and continue with letters, digits and _. They are not case sensitive (`Count` and `COUNT` are the same name). Keywords (Appendix B) cannot be used as names. CPU register names (`R0`–`R31`, `A0`, `SP`, ...) refer to the registers.

3.4 Numbers and characters

Form	Example	Value
Decimal	`123`, `25_000_000`	_ may separate digits
Hexadecimal	`&HFF`, `0xFF`, `&H4000_0000`	255, 255, 1073741824
Binary	`&B1010`, `0b1010`	10
Kilobytes	`8K`, `2K`	8192, 2048 (decimal numbers with a `K` suffix)
Character	`"A"`	A one-character string used as a value is its code: 65
String	`"say ""hi"""`	Text in double quotes; `""` inside stands for one `"`

All values are 32-bit integers. Constant expressions are evaluated at compile time.

4 Target and memory

```
TARGET RV32IC          ' RV32I | RV32IC | RV32E | RV32EC, optional +M, +Zbb
CLOCK  25_000_000      ' Hz; required for DELAY_US / DELAY_MS
CODE   AT &H0000 SIZE 8K
RAM    AT &H8000 SIZE 2K
STACK  256             ' bytes at the top of RAM (default 256)
CPI    1               ' cycles per instruction, for delay loops (default 1)
```

4.1 TARGET

Target	Instruction set
`RV32I`	Base integer instruction set, 32 registers.
`RV32IC`	RV32I with compressed (16-bit) instructions.
`RV32E`	Embedded base set: only `R0`–`R15`.
`RV32EC`	RV32E with compressed instructions.
`+M`	Hardware multiply and divide (`mul`, `div`, `rem`, ...). Without it, `*`, `/` and `MOD` call small runtime routines.
`+Zbb`	Bit-manipulation extension: `ROL` and `ROR` use `rol`, `ror` and `rori`.

Example: `TARGET RV32IC+M+Zbb`. On C targets every instruction that has a 16-bit form is emitted compressed, and branches and jumps are shortened when their targets are in range (section 17.5).

4.2 CLOCK and CPI

`CLOCK` gives the core clock in Hz. It is required when `DELAY_US` or `DELAY_MS` is used; a missing clock is a compile error, not a silent timing bug. `CPI` gives the average cycles per instruction of the delay loop for multi-cycle soft cores (chapter 14).

4.3 CODE, RAM and STACK

- `CODE AT addr SIZE n`: where the program is placed and how big the code memory is. A program larger than `SIZE` is a compile error.
- `RAM AT addr SIZE n`: the data memory. Variables are placed from the start of RAM.
- `STACK n`: bytes reserved at the top of RAM for the stack (default 256). RAM variables plus `STACK` must fit in `RAM SIZE`.

4.4 Startup

The compiler adds a short startup sequence at the `CODE` address (symbol `__start`). It:

1. sets `SP` to the top of RAM and `GP` to RAM base + 2048,
2. clears all RAM variables to 0,
3. sets `mtvec` to the interrupt dispatcher if the program has `ON IRQ` blocks,
4. calls `SUB BoardInit` if the program defines one (section 13.4),
5. starts the BASIC program at the label `main`.

Because `GP` points 2 KB into RAM, every variable in the first 4 KB of RAM is reached with a single `lw`/`sw` instruction and a 12-bit offset.

5 Include files and conditional compilation

5.1 .include

```
.include "lib/uart0.bas"      ' device declarations, CONSOLE, shared SUBs
```

- The lines of the named file replace the `.include` line, as if they were typed there. The program is still one source and one compile.
- The directive stands alone on its line. A `'` comment may follow, a `:` separator may not. `.include` is not case sensitive.
- A relative name is searched first in the directory of the file that contains the `.include`, then in the `-I` directories in order. An absolute path is used as it is.
- Each file is included **once**. A second `.include` of the same file is skipped, so a library can include what it needs. A file that includes itself, directly or through other files, is an error.
- Includes nest. Errors, warnings, the listing and the debugger line table name the included file and its own line numbers.

5.2 #IF, #ELSEIF, #ELSE, #ENDIF

```
#IF NOT DEFINED(BOARD)          ' default when the command line has no -D BOARD=...
CONST BOARD = 1
#ENDIF

#IF BOARD = 1
  .include "board1.bas"
#ELSEIF BOARD = 2
  .include "board2.bas"
#ELSE
  #ERROR "BOARD must be 1 or 2"
#ENDIF
```

- Lines in a branch that is off are not compiled at all. They may contain other definitions of the same names, or text that is not BASIC. An `.include` in such a branch is not read.
- The condition is a constant expression: numbers, `CONST` names defined **above** the `#IF` line (also in included files), operators, and `DEFINED(name)`, which is 1 if a `CONST` of that name exists. A condition is true when it is not 0.
- An unknown name is an error, so a typing error cannot quietly switch code off. Use `DEFINED(name)` when a `CONST` may be missing.
- `#END IF` is the same as `#ENDIF`. Blocks nest; an `#IF` must be closed in the same file.
- `#ERROR "text"` stops the compile with that message when its branch is on.
- Each directive stands alone on its line; a `'` comment may follow.
- The listing shows lines of a branch that is off with `(off)`.

> **NOT on numbers.** `NOT` is logical on a comparison or on `DEFINED()`, but on a number it inverts all bits: `NOT 1` is `&HFFFFFFFE`, which is true. Write `#IF DEBUG = 0`, not `#IF NOT DEBUG`.

5.3 Defining constants on the command line

`-D NAME` or `-D NAME=value` defines `CONST NAME` before the first line (default value 1). One source can then build several variants:

```
python riscbasic.py app.bas -D DEBUG -D BOARD=2
```

6 Objects: registers, variables and constants

Everything that can appear on either side of = is an **object** with a kind, a width and an access method.

Kind	Example	Notes
CPU register	R10, A0, SP	R0 / ZERO is read-only
Register bit	R10.0 or R10[0]	a single bit
Register field	R10[7:4]	bits 7 down to 4, right-aligned when read
Variable	count	in RAM, or pinned to a register
Array element	buf(i)	element of a RAM array
Memory	MEM[addr], MEM8[addr], MEM16[addr]	any 32-bit address (chapter 8)
Device register	UART0.DATA	memory-mapped (chapter 7)
Device bit or field	GPIO.OUT.3, TIMER.CTRL[2:0]	read-modify-write, or set/clear registers if declared
Alias	LED	a name for any object above
Constant	CONST BAUD = 115200	compile time only

Bits and fields can be read and assigned on registers, variables, array elements, device registers and memory: `count.0 = 1`, `buf(i)[7:4] = 5`, `MEM[p].31 = 0`. An assignment to a bit or field of a variable, array element or memory word is a read-modify-write.

6.1 CPU registers

Numeric and ABI names are accepted and fully interchangeable.

Numeric	ABI	Numeric	ABI
R0	ZERO	R8	S0 / FP
R1	RA	R9	S1
R2	SP	R10–R17	A0–A7
R3	GP	R18–R27	S2–S11
R4	TP	R28–R31	T3–T6
R5–R7	T0–T2		

Registers the compiler reserves:

Register	Use	In user code
S0, S1, A5 (R8, R9, R15)	Compiler temporaries. They are in x8–x15, so the compiler's own code can use 16-bit instructions.	Error
GP (R3)	Points into RAM for one-instruction variable access.	Writing it is an error
SP (R2)	Stack pointer.	Writing it gives a warning
RA (R1)	Return address; overwritten by calls, PRINT and runtime routines.	Warning if the program also makes calls
R0 (ZERO)	Always 0.	Read-only
TP (R4)	Return address of the interrupt return (section 13.3).	Error in programs with ON IRQ

T0–T2, A0–A4, A6, A7, S2–S11, T3–T6, and TP in programs without ON IRQ, are free for user variables. On RV32E targets only R0–R15 exist.

6.2 Variables: DIM

```
DIM count AS WORD IN R10    ' pinned to a register
DIM flags AS BYTE           ' in RAM
DIM temp  AS SHALF          ' signed 16-bit, in RAM
DIM buf(16) AS BYTE         ' array of 16 bytes in RAM: buf(0) to buf(15)
DIM name AS STRING * 20     ' text buffer (chapter 11)
```

Type	Width	Range	Notes
`BIT`	1 bit	0, 1	one byte in RAM
`BYTE`	8	0 to 255	
`SBYTE`	8	−128 to 127	signed
`HALF`	16	0 to 65535	
`SHALF`	16	−32768 to 32767	signed
`WORD`	32	0 to 4294967295	default type of parameters and functions
`SWORD`	32	−2147483648 to 2147483647	signed
`STRING * n`	n bytes	text	same as `DIM name(n) AS BYTE`, marked as text

- **RAM variables always live in RAM.** Every read is a load and every write is a store, so the listing is predictable: `count = count + 1` is load, add, store.
- **Register variables** (`IN Rn`) always live in their register. They are the fastest form and the right choice for loop counters and hot values.
- Arrays are in RAM. Elements are numbered from 0; `DIM buf(16)` has `buf(0)` to `buf(15)`. The index is not checked at run time.
- All RAM variables start at 0 (startup clears RAM).
- A `DIM` inside a `SUB` or `FUNCTION` creates a variable that only that routine can see. It is a static RAM variable, not a stack slot (see Appendix C on recursion).

6.3 Constants: CONST

```
CONST BAUD = 115200
CONST CR = 13
CONST LF = 10
CONST MASK = (1 SHL 5) OR 1      ' constant expressions are evaluated at compile time
```

A constant has no storage; it is replaced by its value. A constant may not have the name of a `SUB` or `FUNCTION`.

6.4 Aliases: ALIAS

```
ALIAS LED     = GPIO.OUT.3
ALIAS BUTTON  = GPIO.IN.0
ALIAS RESET_N = NOT GPIO.OUT.7   ' NOT makes an active-low alias (section 7.3)
```

An alias is a second name for an object. It costs nothing at run time: `LED = BUTTON` compiles exactly as `GPIO.OUT.3 = GPIO.IN.0`.

7 Devices

Memory-mapped peripherals are described once with a `DEVICE` block. The program then uses register and bit names instead of addresses and masks.

```
DEVICE GPIO AT &H4000_0000
  REG OUT  AT +0
  REG IN   AT +4  READONLY
  REG SET  AT +8  SETS OUT      ' write 1 = set bit in OUT
  REG CLR  AT +12 CLEARS OUT    ' write 1 = clear bit in OUT
END DEVICE

DEVICE UART0 AT &H4000_1000
  REG DATA   AT +0 : 8
  REG STATUS AT +4 READONLY
  REG RXDATA AT +8 : 8 READONLY
  BIT TXREADY = STATUS.0
  BIT RXREADY = STATUS.1
END DEVICE
```

7.1 Registers, bits and fields

Declaration	Meaning
`REG name AT +offset`	A register at base address + offset. 32 bits wide by default.
`... : 8`, `... : 16`	Register width. Selects `sb`/`sh`/`sw` and `lbu`/`lhu`/`lw`.
`READONLY`	The register cannot be written; an assignment to it is an error.
`WRITEONLY`	The register cannot be read back; the compiler keeps a RAM copy (section 7.4).
`SETS reg`	Writing 1 to a bit of this register sets that bit in `reg`.
`CLEARS reg`	Writing 1 to a bit of this register clears that bit in `reg`.
`BIT name = REG.n`	A name for one bit.
`FIELD name = REG[hi:lo]`	A name for a group of bits.
`NOT` before `REG.n` or `REG[hi:lo]`	Active-low bit or field (section 7.3).

In the program, write `DEVICE.REG`, `DEVICE.BIT` or `DEVICE.REG.n`:

```
UART0.DATA = "A"              ' store one byte
WAIT UNTIL UART0.TXREADY      ' poll a status bit
GPIO.OUT.3 = 1                ' one store to SET, because SETS OUT is declared
GPIO.OUT[11:8] = 5            ' read-modify-write of a field
x = GPIO.IN AND &HFF
```

7.2 Set and clear registers

Many GPIO blocks have separate set and clear registers. When they are declared with `SETS` and `CLEARS`, single-bit assignments become **one store** instead of a read-modify-write, which is also safe against interrupts changing other bits of the register:

Statement	Generated code (RV32I)
`GPIO.OUT.3 = 1` (SET declared)	`lui a5, 0x40000` / `li s0, 8` / `sw s0, 8(a5)`
`GPIO.OUT.3 = 1` (no SET)	`lw` / `ori` / `sw` (read-modify-write)
`LED = BUTTON`	read the bit from `IN`, then store to `SET` or `CLR` depending on its value

7.3 Active-low bits and fields (NOT)

`NOT` in a `BIT`, `FIELD` or `ALIAS` declaration makes it active-low. The program always works with the logical value; the compiler inverts on every read and write.

```
DEVICE PORT AT &H4000_0000
  REG OUT AT +0
  REG SET AT +8  SETS OUT
  REG CLR AT +12 CLEARS OUT
  BIT  LED_N  = NOT OUT.5        ' the LED lights when the pin is low
  FIELD SEL_N = NOT OUT[11:8]
END DEVICE

PORT.LED_N = 1                   ' writes 0 to the pin: one store to CLR
```

- Reading `PORT.LED_N` gives 1 when the hardware bit is 0. A field reads as its bitwise inverse within the field width.
- With `SETS`/`CLEARS` declared, the compiler swaps the two registers, so it is still one store.
- `IF`, `WHILE`, `WAIT UNTIL` and `LOOP UNTIL` test the hardware bit the other way round at no extra cost.
- A device whose `TXREADY` is declared with `NOT` makes `PRINT` wait while the hardware bit is 1.
- `NOT` twice cancels: `ALIAS X = NOT PORT.LED_N` is the raw pin. A whole register cannot be inverted.

7.4 Write-only registers (WRITEONLY)

Some output registers cannot be read back: a read returns 0, garbage or the input pins. `WRITEONLY` makes the compiler keep a copy of the register in RAM, the *shadow*:

```
DEVICE LEDS AT &H4000_2000
  REG OUT AT +0 WRITEONLY
  BIT LED = OUT.3
END DEVICE

LEDS.LED = 1              ' set bit 3 in the shadow, then write the whole shadow to OUT
PRINT HEX LEDS.OUT        ' reads the shadow, not the hardware
```

- Every write goes to the shadow and to the register. Every read, including the read-modify-write behind bit and field assignments, uses the shadow.
- Writes through a `SETS` or `CLEARS` register also update the shadow.
- The shadow is a RAM variable created only when the register is used. It starts at 0; if the hardware resets to another value, write the whole register once at the start.
- It appears in the ELF symbol table as `DEVICE.REG.shadow`. `MEM[addr]` access does not use it.
- `READONLY` and `WRITEONLY` together are an error.

8 Memory access: MEM

```
MEM[&H4400_0000] = &H1234_5678     ' write a 32-bit word anywhere in the 4 GB space
v = MEM[&HFFFF_FFFC]               ' read the top word
MEM[base + 4 * i] = v              ' the address can be any expression
MEM8[p] = 13 : x = MEM16[p + 2]    ' byte and halfword access
MEM[p][15:8] = &HAB                ' a field of a memory word (read-modify-write)
IF MEM[p].31 THEN PRINT "set"      ' a bit
```

- `MEM[addr]` is a 32-bit word. `MEM8`, `MEM16` and `MEM32` give the width explicitly. Reads of `MEM8` and `MEM16` are zero-extended.
- The address is a full 32-bit value: a constant or any expression. Parentheses also work: `MEM32(addr)` is the same as `MEM32[addr]`.
- The first `[ ]` after `MEM` is the address; a second `[hi:lo]` or `.n` selects bits.
- The address is not checked for alignment. A misaligned `MEM16` or `MEM` access does whatever the core does with it (trap or split access).
- `MEM`, `MEM8`, `MEM16` and `MEM32` are reserved words.

8.1 Cost of an access

Address	Code
Constant, in the first 4 KB of RAM	one `lw`/`sw` relative to `GP`
Constant, within 2 KB of address 0 or in the top 2 KB (`&HFFFF_F800`–`&HFFFF_FFFF`)	one `lw`/`sw` relative to `zero`
Any other constant	`lui a5, hi` + `lw`/`sw` with the low 12 bits as offset
Computed	the address expression is evaluated into a register first

Back-to-back accesses with the same upper 20 address bits share one `lui`. This applies to device registers too. A label, a call or any other change of `A5` ends the sharing:

```
; MEM[&H4400_0000] = 1
    c.li    s0, 1
    lui     a5, 0x44000
    c.sw    s0, 0(a5)
; MEM[&H4400_0004] = 2
    c.li    s0, 2
    c.sw    s0, 4(a5)        ; no second lui
```

9 Assignment and expressions

9.1 The assignment rule

`DEST = SOURCE` is compiled by looking at the pair (destination kind, source kind). The compiler chooses the shortest correct sequence:

Statement	Generated code (RV32I)
`R1.0 = 1`	`ori x1, x1, 1`
`R1.0 = 0`	`andi x1, x1, -2`
`R1.12 = 1`	`lui a5, 1` / `or x1, x1, a5` (the mask does not fit in 12 bits)
`R10 = R11.3`	`srli x10, x11, 3` / `andi x10, x10, 1`
`R10[7:4] = 5`	`andi x10, x10, -241` / `ori x10, x10, 0x50`
`count = count + 1` (RAM)	`lw s0, ofs(gp)` / `addi s0, s0, 1` / `sw s0, ofs(gp)`
`UART0.DATA = "A"`	`li s0, 65` / `sb s0, 0(a5)`
`LED = count > 10`	the comparison result (0 or 1) is assigned as a bit

- **Width:** a wider source is truncated to the destination width; a narrower source is zero-extended, or sign-extended for signed types. Truncating a constant gives a warning.
- **Bits:** any source assigned to a bit is true if it is not zero.
- **Fields:** the value is masked to the field width and shifted into place.
- **Read-only destinations** (`R0`, `READONLY` registers, constants, `STRING` parameters) are errors.

9.2 ATOMIC

```
ATOMIC GPIO.OUT[3:0] = mode
```

`ATOMIC` before an assignment disables interrupts (`mstatus.MIE`) around it and restores the previous state afterwards. Use it for read-modify-write of registers that an interrupt handler also changes.

9.3 Operators

From highest to lowest precedence:

Precedence	Operators	Meaning
1	`-x`, `+x`	negation
2	`*`, `/`, `MOD`	multiply, divide, remainder
3	`+`, `-`	add, subtract
4	`SHL`, `SHR`, `SAR`	shift left, shift right logical, shift right arithmetic
5	`=`, `<>`, `<`, `>`, `<=`, `>=`	comparison; result 0 or 1
6	`NOT`	logical NOT of a comparison, otherwise bitwise inversion
7	`AND`	bitwise AND (logical on comparisons)
8	`OR`, `XOR`	bitwise OR and XOR

- Use parentheses to group: `(a + b) * c`.
- Comparisons, `/` and `MOD` are **unsigned** unless one operand is signed (`SBYTE`, `SHALF`, `SWORD`); then they are signed: `-7 / 2` is `-3`, `-7 MOD 2` is `-1`.
- `*`, `/` and `MOD` use M-extension instructions with `+M`. Otherwise the compiler includes a small runtime routine the first time one is used.
- Constant expressions are folded at compile time, so `CONST`s and literal arithmetic cost nothing.
- A one-character string is a number: `IF c = "A" THEN` compares `c` with 65.

10 Shift and rotate commands

Besides the SHL, SHR and SAR operators, four commands shift or rotate a register or RAM variable in place:

```
SHL dest [, count]     ' shift left, zeros in
SHR dest [, count]     ' shift right, zeros in (logical)
ROL dest [, count]     ' rotate left
ROR dest [, count]     ' rotate right
```

- `dest` is a CPU register, a register variable or a RAM variable. For device registers, use an assignment instead.
- `count` defaults to 1. It is a constant, a CPU register or a register variable. Variable counts are allowed only for 32-bit destinations.
- The operation works on the **width of the destination**: a `BYTE` rotates within 8 bits, a `HALF` within 16.
- RV32I has no rotate instructions, so `ROL`/`ROR` are built from two shifts and an `or`. With `+Zbb` the compiler uses `rol`, `ror` and `rori`.

Command	Generated code (RV32I)
`SHL R10, 3`	`slli x10, x10, 3`
`SHR R10, R11`	`srl x10, x10, x11`
`SHL count, 2` (RAM `WORD`)	`lw s0, ofs(gp)` / `slli s0, s0, 2` / `sw s0, ofs(gp)`
`ROL R10, 4`	`slli s0, x10, 4` / `srli x10, x10, 28` / `or x10, x10, s0`
`ROL R10, 4` (`+Zbb`)	`rori x10, x10, 28`
`ROR flags` (RAM `BYTE`)	`lbu s0, ofs(gp)` / `srli s1, s0, 1` / `slli s0, s0, 7` / `or s0, s0, s1` / `sb s0, ofs(gp)`

11 Strings

RiscBasic has no heap. Every string is a fixed-size, zero-terminated `BYTE` buffer.

```
DIM name AS STRING * 20               ' 20 bytes: up to 19 characters and the 0

FUNCTION Hex8(v AS WORD) AS STRING * 12
  RETURN "0x" + HEX$(v)
END FUNCTION

FUNCTION Greet(who AS STRING) AS STRING * 32
  RETURN "Hello, " + who + "!"
END FUNCTION

name = "item " + DEC$(n) + " at " + Hex8(addr) + CHR$(33)
PRINT Greet(name), 13, 10
```

11.1 Buffers and string expressions

- `DIM name AS STRING * n` is the same as `DIM name(n) AS BYTE`, but marks the array as text. Any `BYTE` array can be used as a string.
- A **string expression** joins terms with `+`. A term is `"text"`, a `STRING` or `BYTE` array, a string `FUNCTION` call, a `STRING` parameter, `CHR$(x)`, `DEC$(x)`, `HEX$(x)`, or a substring (`LEFT$`, `RIGHT$`, `MID$`).
- `buffer = string expression` builds the text in the buffer. It is cut at the buffer size and always zero-terminated. `s = "x" + s` works: when the buffer itself is read during the build, the text is built in a scratch buffer and then copied.
- Strings cannot be used as numbers, and numbers cannot be assigned to strings: use `DEC$`, `HEX$`, `CHR$` and `VAL`.

11.2 String functions

Function	Result
`CHR$(x)`	one character with code `x`
`DEC$(x)`	`x` in decimal, as `DEC` in `PRINT` (signed types with `-`)
`HEX$(x)`	`x` in hex digits, as `HEX` in `PRINT`
`LEFT$(s, n)`	the first `n` characters
`RIGHT$(s, n)`	the last `n` characters
`MID$(s, start [, n])`	`n` characters from position `start` (from 1); without `n`, to the end
`LEN(s)`	number of characters (a number)
`INSTR([start,] s, find)`	position of `find` in `s` from 1; 0 if not found (a number)
`VAL(s)`	s converted to a signed number (a number)

- Out-of-range positions give shorter or empty text, never an error: `MID$(s, 0)` and a `start` past the end are empty.
- The source of `LEFT$`, `RIGHT$`, `MID$`, `INSTR` and `VAL` is a single term; nest them through a variable. `PRINT` and `LEN` use a substring directly.
- `LEN` of joined strings adds the lengths of the terms without building them. `LEN(DEC$(x))` is not supported.
- `VAL` skips leading spaces, accepts `+` or `-`, decimal digits, or hex/binary digits after `&H`, `0x`, `&B`, `0b`. The first character that is not a digit ends the number: `VAL("42abc")` is 42. No digits gives 0.
- With constant text, `LEN`, `INSTR`, `VAL` and the substring functions are computed at compile time.

11.3 Comparing strings

```
IF name = "Bob" THEN PRINT "hi Bob", 13, 10
IF LEFT$(cmd, 3) = "SET" THEN v = VAL(MID$(cmd, 5))
```

- `=`, `<>`, `<`, `>`, `<=`, `>=` compare byte by byte, unsigned. A shorter prefix is less: `"ab" < "abc"`.
- Each side is one term. To compare a joined string, assign it to a buffer first.
- Only one side can be a substring, and both sides cannot call the same string `FUNCTION` (it has one result buffer).

11.4 String functions and parameters

- `FUNCTION f(...) AS STRING * n` returns text in its own result buffer of `n` bytes. `RETURN expression` sets it. Ending without `RETURN`, or with `EXIT FUNCTION`, returns empty text.
- Each call overwrites the previous result. Using results one after another (`Pad(7) + "/" + Pad(42)`) and nesting (`Greet(Greet("x"))`) work; a recursive string `FUNCTION` does not.
- A `STRING` parameter (`who AS STRING`) passes a pointer to existing text: a literal, a buffer, a string function result or another `STRING` parameter. It is read-only. The argument is a single term.
- Do not build strings in an interrupt handler while the main program may be building one: the build state is shared.

12 Control flow

12.1 IF

```
IF temp > 80 THEN
  FAN = 1
ELSEIF temp < 60 THEN
  FAN = 0
ELSE
  PRINT "hold", 13, 10
END IF

IF ok THEN PRINT "a" : PRINT "b" ELSE PRINT "c"     ' single-line form
```

- The block form ends with `END IF`.
- The single-line form takes all statements after `THEN` up to its `ELSE`, and all statements after the `ELSE`. An `END IF` on such a line is an error.
- An `ELSE` belongs to the nearest `IF` that has no `ELSE` yet: in `IF a THEN IF b THEN x = 1 ELSE x = 2`, the `ELSE` runs when `a` is true and `b` is false.
- A condition is true when it is not 0. Bit objects (`IF UART0.RXREADY THEN`) are tested with a single branch.

12.2 Loops

```
WHILE cond ... WEND
DO ... LOOP                    ' endless
DO ... LOOP UNTIL cond
REPEAT ... UNTIL cond          ' body runs at least once
FOR i = 0 TO 15 [STEP 1] ... NEXT
```

- `REPEAT ... UNTIL cond` is the same as `DO ... LOOP UNTIL cond`. `UNTIL` on its own line closes the innermost `REPEAT`.
- `EXIT FOR`, `EXIT DO`, `EXIT WHILE` and `EXIT REPEAT` leave the innermost loop of that kind.

12.2.1 FOR loop counters

```
FOR i = 0 TO 15 ... NEXT          ' RAM variable: load / add / store each pass
FOR ticks = 0 TO 99 ... NEXT      ' register variable (DIM ticks AS WORD IN A0)
FOR R12 = 0 TO 15 ... NEXT        ' CPU register, by number
FOR S2 = 10 TO 0 STEP -1 ... NEXT ' CPU register, by ABI name
```

- Register counters are the fastest form: the loop is `addi` plus one compare-and-branch. With constant bounds, the end value is used as an immediate where it fits.
- `STEP` may be negative. The counter is compared signed.
- `S0`, `S1`, `A5`, `GP` and `R0` cannot be loop counters.

12.3 SELECT CASE

```
SELECT CASE key
  CASE "a" TO "z"
    PRINT "lower", 13, 10
  CASE 1, 2, 10 TO 12
    PRINT "control", 13, 10
  CASE IS > 127
    PRINT "high", 13, 10
  CASE ELSE
    PRINT "other", 13, 10
END SELECT
```

- `CASE` takes constants and constant ranges, separated by commas. A range includes both ends; `CASE 5 TO 1` (low end last) is an error.

- `CASE IS op value` compares with `=`, `<>`, `<`, `>`, `<=` or `>=`.
- Comparisons are signed if the `SELECT` expression is signed, otherwise unsigned.
- The first matching `CASE` runs; there is no fall-through.

12.4 GOTO, labels and END

```
retry:
  n = n + 1
  IF n < 3 THEN GOTO retry
END                             ' stop: endless loop at __end
```

A label is a name followed by `:` at the start of a line. `END` on its own stops the program; reaching the last line has the same effect.

13 Subroutines, functions and interrupts

13.1 SUB and FUNCTION

```
SUB Send(ch AS BYTE)            ' parameters in A0-A4, A6, A7
  WAIT UNTIL UART0.TXREADY
  UART0.DATA = ch
END SUB

FUNCTION Add(a AS WORD, b AS WORD) AS WORD
  RETURN a + b
END FUNCTION

Send 65                         ' call forms: Send 65, Send(65), CALL Send(65)
total = Add(total, 5)
```

- Parameters have a type (`AS WORD` is the default). A `FUNCTION` has a return type (default `WORD`) or `AS STRING * n` (chapter 11).
- `RETURN value` leaves a `FUNCTION` with a value. `EXIT SUB` and `EXIT FUNCTION` leave early.
- A `SUB` or `FUNCTION` may be defined before or after its first call, and anywhere in the program; the main program jumps around it.
- Variables declared with `DIM` inside the routine are local to it (static RAM).

13.2 Calling convention

- Arguments go in `A0`–`A4`, `A6`, `A7` (up to 7; on RV32E `A0`–`A4`, up to 5). `A5` is skipped because it is a compiler temporary. `A0` holds the return value.
- **Register variables are global.** A `SUB` or `FUNCTION` sees and may change every register variable, and the callee saves only `RA`. This differs from the standard RISC-V ABI.
- A global register variable that sits in an argument register is saved by the caller around each call that uses that register. Inside a `SUB`, it is hidden by the parameter in the same register; using it there is an error.
- Recursion is allowed, with the limits in Appendix C.

13.3 Interrupts: ON IRQ

```
TARGET RV32IC
CLOCK  25_000_000
CODE   AT &H0000 SIZE 8K
RAM    AT &H8000 SIZE 2K

DIM ticks AS WORD

ON IRQ MTIMER                   ' machine timer interrupt (mcause 7)
  ticks = ticks + 1
  ' acknowledge / re-arm the timer here (core specific)
END IRQ

ASM
  li     t0, 128
  csrrs  zero, mie, t0          ' enable the machine timer interrupt
  csrrsi zero, mstatus, 8       ' global interrupt enable
END ASM

DO
LOOP
```

- `ON IRQ n` takes an interrupt number (0–2047) or one of the names `MSOFT` (3), `MTIMER` (7) and `MEXT` (11). For other names, define a `CONST`.

- The compiler builds one dispatcher at `mtvec`: it saves the caller-saved registers, reads `mcause` and calls the matching `ON IRQ` block.
- Interrupts are not enabled by the compiler. Enable them with inline assembly as above.
- The dispatcher returns without `mret`: it restores the registers, loads the return address into `TP`, sets `mstatus.MIE` and jumps. Some cores execute `mret` as a no-op; the AMD MicroBlaze V MCS does. `TP` is therefore reserved in programs with `ON IRQ`.
- Use `ATOMIC` for read-modify-write of data that a handler also changes.

13.4 Board initialisation: SUB BoardInit

```
SUB BoardInit
  GPIO.OUT = 0                    ' known output state
  PLL.CTRL = PLL_CONFIG
END SUB
```

If the program has a `SUB` named `BoardInit`, the startup code calls it once, after `SP`, `GP` and RAM clearing are set up and before the `main` label, where the BASIC program starts. A debugger that stops at `main` therefore sees the board already initialised.

- `BoardInit` may be defined anywhere: after the main program, in an included file, or inside an `#IF` branch, so each board file can bring its own.
- It must be a `SUB` without parameters.
- Interrupts are not yet enabled when it runs.
- A program that also calls it with `CALL` runs it twice.

14 Timing and waiting

```
WAIT UNTIL UART0.TXREADY        ' poll until the condition is true
DELAY_US 10                     ' busy-wait 10 microseconds
DELAY_MS 100                    ' busy-wait 100 milliseconds
```

- `WAIT UNTIL cond` repeats the test until it is true. A bit test is one load and one branch.
- `DELAY_US` and `DELAY_MS` are busy-wait loops calculated from `CLOCK`. They require `CLOCK`.
- The loop is calculated as 2 instructions per pass, times `CPI`. On a multi-cycle soft core, set `CPI` to its average cycles per instruction and check the timing on real hardware.
- Interrupt handlers that run during a delay make it longer.

15 Console input and output

15.1 CONSOLE

`CONSOLE dev` names the device used by `PRINT`, `INPUT` and `LINE INPUT` without an explicit device. It applies to the statements after it and may be given again to switch devices.

A device used for output must declare a register `DATA` and a bit `TXREADY`. A device used for input must declare a bit `RXREADY` and a register `RXDATA` (or receive through `DATA`). `NOT` is allowed on both bits.

15.2 PRINT

```
CONST CR = 13
CONST LF = 10
CONSOLE UART0

PRINT "count = ", DEC count, CR, LF
PRINT "addr ", HEX addr, " key ", CHR key, CR, LF
PRINT UART1, "to the other UART", CR, LF
PRINT "Hello, ", name, CR, LF        ' BYTE array: printed as text
```

Item	Output
`"text"`	the text
number or `CONST` (0–255)	that one byte, for example `13, 10` for a line end
`CHR expr`	the low 8 bits of `expr` as one character
`HEX expr`	hex digits, uppercase, no prefix; `BYTE` 2 digits, `HALF` 4, `WORD` and registers 8, with leading zeros
`DEC expr`	decimal without leading zeros; signed types print `-` when negative
`BYTE` array name	the array as zero-terminated text, never past its end
string expression	any string expression from chapter 11

- Items are separated by `,` (or `;`). **No line end is added**: write `13, 10` where a line ends.
- A device name directly after `PRINT`, followed by `,` or the end of the line, selects the device.
- A variable on its own is an error: write `CHR`, `DEC` or `HEX`.
- Adjacent text and character constants are merged at compile time: `"done", 13, 10` is stored once as one string.
- `DEC` uses a table of powers of ten, so it needs no multiply or divide and works on plain RV32I.

15.3 INPUT

```
INPUT "Value? ", DEC v          ' prompt, then a decimal number
INPUT CHR c                     ' wait for one character (no echo)
INPUT UART1, HEX addr           ' explicit device, hexadecimal number
IF UART0.RXREADY THEN INPUT CHR c   ' only if a character is waiting
```

- Items: `"prompt"`, `CHR target`, `DEC target`, `HEX target`. The target is anything assignable.
- `DEC` reads an optional `-` and decimal digits; `HEX` reads `0`–`9`, `A`–`F`, `a`–`f`. Both echo the characters they accept. The first other character ends the number and is not echoed. No digits gives 0.
- `INPUT` waits for a character. To check without waiting, test `RXREADY`.

15.4 LINE INPUT

```
DIM name(20) AS BYTE
DIM n AS WORD
LINE INPUT "Name? ", name, n     ' text into name(), length into n
PRINT "Hello, ", name, 13, 10
```

- Form: `LINE INPUT [dev,] ["prompt",] buffer [, length]`. `buffer` is a `BYTE` array or `STRING`.
- The text is zero-terminated: a buffer of N bytes holds up to N−1 characters.
- Enter (CR) ends the line and is not stored. LF is ignored, so terminals that send CR LF work.
- Backspace (8) and DEL (127) remove the last character and erase it on the terminal. Other control characters are ignored. When the buffer is full, further characters are ignored until Enter.

15.5 READ and WRITE

`READ` and `WRITE` send or receive a list of values through routines the program defines. The name before the first comma selects them: `WRITE dev, ...` calls `SUB devWrite`, `READ dev, ...` calls `FUNCTION devRead`.

```
SUB SpiWrite(v AS BYTE)          ' one value out
  WAIT UNTIL SPI.TXREADY
  SPI.DATA = v
END SUB

FUNCTION SpiRead AS BYTE         ' one value in
  WAIT UNTIL SPI.RXREADY
  RETURN SPI.DATA
END FUNCTION

WRITE Spi, &H03, addr SHR 8, addr      ' three calls of SpiWrite
READ Spi, id, buf(0), buf(1)           ' three calls of SpiRead
```

- Form: `WRITE dev, item, ...` and `READ dev, target, ...`, with at least one item. `dev` is only a name: it may be a declared `DEVICE`, but does not have to be. Case does not matter.
- `WRITE` needs `SUB devWrite` with exactly one parameter (a `FUNCTION` also works; its result is ignored). Items are expressions, or string arguments when the parameter is a `STRING`.
- `READ` needs `FUNCTION devRead` without parameters, returning a number. Each target receives the result of one call, as in an assignment: variables, registers, array elements, device registers, bits and fields.
- Items are handled left to right, one call each: `WRITE Spi, a, b` is `SpiWrite a : SpiWrite b`.
- A missing or unsuitable routine is a compile error that names the routine expected.

An item can also name a procedure of the device: a lone name `X` for which `SUB devX` exists calls that SUB, without a value, at its place in the list.

```
SUB SpiStart                     ' chip select active
  SPI.CS = 1
END SUB

SUB SpiStop                      ' chip select inactive
  SPI.CS = 0
END SUB

WRITE Spi, Start, &H03, addr SHR 8, addr     ' SpiStart, then three SpiWrite calls
READ Spi, id, buf(0), Stop                   ' three SpiRead calls, then SpiStop
WRITE Spi, Start, &H06, Stop                 ' SpiStart : SpiWrite &H06 : SpiStop
```

- A procedure item is a single name followed by `,` or the end of the statement, with a `SUB devName` that has no parameters (a `FUNCTION` without parameters also works; its result is ignored).
- If the name is also a variable, constant, alias, device, register or routine, the item is ambiguous: a compile error asks to rename one.
- `devWrite` / `devRead` is needed only when the statement also has value items: `WRITE Spi, Start` alone is allowed.

16 Inline assembly

```
ASM
  csrrsi zero, mstatus, 8        ' global interrupt enable
  li     t0, 128
  csrrs  zero, mie, t0
END ASM
```

- `ASM` ... `END ASM` contains RISC-V instructions, one per line, with ABI or numeric register names.
- Variables and aliases can be used by name inside `ASM` blocks.
- The assembler keeps exactly the instructions written; the compiler does not optimise them.
- Do not change `GP`, `SP` or the compiler temporaries `S0`, `S1`, `A5` in a way the surrounding code does not expect.

17 Compiler output and debugging

17.1 Output files

File	Contents
`.bin`	Raw image from the `CODE` address.
`.hex`	Intel HEX.
`.mem`	One 32-bit word per line, for `$readmemh`.
`.lst`	Listing (section 17.2). Written by default; `--no-lst` turns it off.
`.elf`	ELF32 RISC-V executable (section 17.3). `--no-elf` turns it off.

The code must fit in `CODE SIZE`, otherwise the compile fails. The size of every `SUB`, `FUNCTION` and runtime routine is reported in the listing.

17.2 The listing file

- The file starts with the license header and the target summary.
- Each source line appears as a comment with its line number, directly above the instructions generated for it. Lines from an included file carry the file name: `; lib/uart0.bas:0018  PRINT CR, LF`.
- Each instruction line shows the address, the machine code in hex (4 digits for compressed instructions) and the disassembly.
- Startup code, runtime routines and string constants are listed under their own headings.
- A summary at the end gives code size per routine, total code used, and RAM used.

```
; ---- program ----
                  main:
; 0048  MEM[&H4400_0000] = 1
00000024      4405   c.li    s0, 1
00000026  440007B7   lui     a5, 0x44000
0000002A      C380   c.sw    s0, 0(a5)
```

17.3 ELF and DWARF

The `.elf` file loads directly into GDB, OpenOCD, Spike, Renode or a simulator testbench, and `objdump -d` disassembles it with all names. It contains:

- one loadable segment with code and constant data at the `CODE` address, entry point `__start`;
- `.bss` describing the RAM variables (cleared by the startup code);
- a symbol table: `__start`, `main`, every `SUB`, `FUNCTION` and runtime routine with its size, IRQ handlers, labels, strings, and RAM variables with their sizes (`SUB.local` for locals);
- DWARF 3 line information that maps every instruction to its `.bas` line, including included files;
- the ISA string in `.riscv.attributes`, for example `rv32i2p1_m2p0_c2p0_zicsr2p0`.

17.4 Source-level debugging

With the DWARF information, debuggers show the `.bas` file and step one BASIC line at a time. Keep the `.bas` files where they were compiled, or tell the debugger where they are.

GDB (for example `riscv-none-elf-gdb program.elf`, connected to the target):

```
break main
break program.bas:34
break lib/uart0.bas:18
next
list
x/wd &PRESSES
```

XSDB (AMD Vitis), after `connect`, `target` and `dow program.elf`:

```
bpadd -file program.bas -line 34
con
nxt
```

Variable names are upper case in the ELF. Variables are available as addresses (`&PRESSES`); the DWARF information covers source lines, not types.

17.5 Compressed instructions and code size

On `RV32IC` and `RV32EC`, every instruction with a 16-bit form is emitted compressed. Code is generated in one pass; at the end, branches and jumps are shortened to 16-bit forms when their targets are in range, and conditional branches beyond ±4 KB become an inverted branch over a `jal`.

Compiler temporaries are `s0`, `s1` and `a5`, all in x8–x15, so the compiler's own code can use the 16-bit forms. Runtime routines are called with `c.jal` where possible.

18 Complete example

```
TARGET RV32IC
CLOCK  25_000_000
CODE   AT &H0000 SIZE 8K
RAM    AT &H8000 SIZE 2K

DEVICE GPIO AT &H4000_0000
  REG OUT  AT +0
  REG IN   AT +4  READONLY
  REG SET  AT +8  SETS OUT
  REG CLR  AT +12 CLEARS OUT
END DEVICE

DEVICE UART0 AT &H4000_1000
  REG DATA   AT +0 : 8
  REG STATUS AT +4 READONLY
  BIT TXREADY = STATUS.0
END DEVICE

ALIAS LED    = GPIO.OUT.3
ALIAS BUTTON = GPIO.IN.0

DIM presses AS WORD          ' RAM
DIM ticks   AS WORD IN A0    ' register

CONSOLE UART0
PRINT "RiscBasic ready", 13, 10

DO
  LED = BUTTON
  IF BUTTON THEN presses = presses + 1
  ticks = ticks + 1
  PRINT "presses ", DEC presses, " ticks ", HEX ticks, 13, 10
  DELAY_MS 100
LOOP
```

The LED follows the button, button presses are counted in a RAM variable, a loop counter runs in register `A0`, and a status line is printed every 100 ms. Compiled for RV32IC, the program is 436 bytes of code, including the print routines.

Appendix A Keyword reference

Keyword	Kind	Purpose	Section
`.include "file"`	Directive	Insert another source file	5.1
`#IF #ELSEIF #ELSE #ENDIF`	Directive	Conditional compilation	5.2
`#ERROR "text"`	Directive	Stop the compile with a message	5.2
`ALIAS`	Declaration	Second name for an object	6.4
`AND OR XOR NOT`	Operator	Bitwise / logical operators	9.3
`ASM ... END ASM`	Statement	Inline assembly	16
`ATOMIC`	Statement prefix	Assignment with interrupts disabled	9.2
`BIT FIELD`	Device	Named bit or field of a device register	7.1
`CALL`	Statement	Call a SUB	13.1
`CASE IS SELECT`	Statement	Multi-way branch	12.3
`CHR DEC HEX`	PRINT / INPUT item	Character, decimal, hex formatting	15
`CHR$ DEC$ HEX$`	String function	Number to text	11.2
`CLOCK CPI`	Declaration	Core clock and cycles per instruction	4.2
`CODE RAM STACK`	Declaration	Memory layout	4.3
`CONSOLE`	Declaration	Default device for PRINT and INPUT	15.1
`CONST`	Declaration	Named constant	6.3
`DELAY_US DELAY_MS`	Statement	Busy-wait delay	14
`DEVICE ... END DEVICE`	Declaration	Memory-mapped peripheral	7
`DIM ... AS ... IN`	Declaration	Variable or array	6.2
`DO ... LOOP [UNTIL]`	Statement	Loop	12.2
`END`	Statement	Stop the program; also ends blocks	12.4
`EXIT`	Statement	Leave FOR, DO, WHILE, REPEAT, SUB, FUNCTION	12.2
`FOR TO STEP NEXT`	Statement	Counted loop	12.2
`FUNCTION ... END FUNCTION`	Declaration	Function with a return value	13.1
`GOTO`	Statement	Jump to a label	12.4
`IF THEN ELSEIF ELSE END IF`	Statement	Conditional	12.1
`INPUT`	Statement	Read a character or number	15.3
`INSTR LEN VAL`	Function	Search, length, text to number	11.2
`LEFT$ RIGHT$ MID$`	String function	Substrings	11.2
`LET`	Statement	Optional assignment keyword	3.2
`LINE INPUT`	Statement	Read a line of text	15.4
`MEM MEM8 MEM16 MEM32`	Object	Memory at any 32-bit address	8
`MOD`	Operator	Remainder	9.3
`ON IRQ ... END IRQ`	Declaration	Interrupt handler	13.3
`PRINT`	Statement	Output to a device	15.2
`READ`	Statement	Values from FUNCTION devRead into targets	15.5
`READONLY WRITEONLY`	Device	Register access restrictions	7.1
`REG ... AT`	Device	Device register	7.1
`REM`	Comment	Comment to the end of the line	3.2
`REPEAT ... UNTIL`	Statement	Loop, tested at the end	12.2
`RETURN`	Statement	Return from a FUNCTION (with value) or SUB	13.1
`SETS CLEARS`	Device	Set / clear registers	7.2
`SHL SHR SAR`	Operator	Shifts	9.3
`SHL SHR ROL ROR`	Statement	Shift / rotate in place	10
`STRING * n`	Type	Text buffer	11.1
`SUB ... END SUB`	Declaration	Subroutine	13.1
`TARGET`	Declaration	Instruction set	4.1
`WAIT UNTIL`	Statement	Poll until a condition is true	14
`WHILE ... WEND`	Statement	Loop, tested at the start	12.2
`WRITE`	Statement	Values to SUB devWrite, one call each	15.5

Appendix B Reserved words

These words cannot be used as names of variables, constants, aliases, labels, SUBs or FUNCTIONs:

ALIAS, AND, AS, ASM, AT, ATOMIC, BIT, CALL, CASE, CHR, CHR$, CLEARS, CLOCK, CODE, CONSOLE, CONST, CPI, DEC, DEC$, DELAY_MS, DELAY_US, DEVICE, DIM, DO, ELSE, ELSEIF, END, EXIT, FIELD, FOR, FUNCTION, GOTO, HEX, HEX$, IF, IN, INPUT, INSTR, IRQ, IS, LEFT$, LEN, LET, LINE, LOOP, MEM, MEM16, MEM32, MEM8, MID$, MOD, NEXT, NOT, ON, OR, PRINT, RAM, READ, READONLY, REG, REPEAT, RETURN, RIGHT$, ROL, ROR, SAR, SELECT, SETS, SHL, SHR, SIZE, STACK, STEP, STRING, SUB, TARGET, THEN, TO, UNTIL, VAL, WAIT, WEND, WHILE, WRITE, WRITEONLY, XOR

Also reserved: the register names R0–R31 and the ABI names ZERO, RA, SP, GP, TP, T0–T6, S0–S11, FP, A0–A7.

Appendix C Implementation notes and limits

- **Recursion.** Parameters and local `DIM` variables are not saved across a recursive call (locals are static RAM). Recursion works only if the routine does not need its parameters after the recursive call.
- **Stack checking.** RAM variables plus `STACK` must fit in `RAM SIZE`. The call depth is not analysed.
- **FOR counters** are compared signed, so a loop must not cross from `&H7FFF_FFFF` to `&H8000_0000`.
- **DELAY_US / DELAY_MS** assume 2 instructions per loop pass times `CPI`. Calibrate `CPI` on real hardware.
- **IRQ names.** `ON IRQ` knows `MSOFT`, `MTIMER` and `MEXT`. Other sources need a number or a `CONST`.
- **Array indexes** are not checked at run time.
- **MEM alignment** is not checked.
- **INPUT** has no line editing; use `LINE INPUT` for editable text.

C.1 Runtime routines

These routines are emitted once, after the program, only if used. They are called with `jal ra` (`c.jal` on C targets), take arguments in `s0`/`s1`, and save every other register they use.

Routine	Used for
`__mul`, `__divu`, `__div`	`*`, `/`, `MOD` without `+M`
`__putc_dev`, `__puts_dev`, `__putbuf_dev`	PRINT of characters, text and BYTE arrays
`__puthex_dev`, `__putdec_dev`	PRINT HEX and DEC
`__getc_dev`, `__getdec_dev`, `__gethex_dev`, `__getline_dev`	INPUT and LINE INPUT
`__sbegin`, `__sapp_chr`, `__sapp_str`, `__sapp_dec`, `__sapp_hex`	building strings
`__strlen`, `__strcmp`, `__sapp_mid`, `__lenmid`, `__putmid_dev`, `__instr`, `__val`	string functions